평가 제출용 | 자료 기준 2026.10.20 | 문서 v1.1

Jetson 실행·ONNX/TensorRT 변환 절차서

엔진 모형 조립 검사 프로젝트 · 실행 코드와 변환 모델의 연결

핵심 구현 성과
학습 PT → ONNX → TensorRT → Jetson 운영 패키지를 해시로 연결했다. 카메라·검출·3 프레임 판정·결과 저장·PLC 응답·웹 조회를 하나의 서비스로 구성하고, 저장 완료를 확인한 뒤 PLC 결과를 전달한다.

확인 항목	문서의 기준
평가 근거 소스	cfaa372eece0cd6a7dc659cc316c256fb4b6af76 Engine_Assembly_Defect_Inspector 의 기존 확인 코드
기준 운영 릴리스	engine-integrated-32c484ac6aeb07f1
실물 7 회 시험 실행본	engine-integrated-4c4b935f8ab0404a 후속 32c4 의 UI 변경 기록과 구분
관측 Jetson 환경	Ubuntu 22.04 · L4T 36.4.7 · Python 3.10.12 CUDA 12.6 · TensorRT 10.3
운영 검출 모델	M1 dynamic_parts_005 / M2 envelope_v007_fp16
대표 최적화 성과	TensorRT 프로세스 RSS 관측 최대 M1 49.1%, M2 48.6% 감소

이 문서는 실행 코드·모델 계약·저장 및 PLC 전달 순서를 설명한다. 표의 기준 커밋은 평가 근거 소스이며, README 변경 커밋이나 장치 릴리스 ID 와 같은 값이 아니다.

바로 확인할 내용	절 / 쪽
운영 흐름과 코드 맵	01-02 절 · 2-3 쪽
모델 계보·변환·입출력·실행	03-08 절 · 4-9 쪽
설치·측정 결과·적용 범위	09-11 절 · 10-12 쪽
실행 소스·세부 보고서 링크	부록 A-B · 13-14 쪽

근거: [G1-G9] 평가 근거 소스, [R03-R06] 기존 모델·Benchmark·통합시험·DB 보고서.

01 현재 Jetson 운영 구조

운영 경로는 /home/jetson/oned_device_bench/current 이다. engine-inspector.service 가 통합 런타임을 감독하고, 장치 설정은 config/, 모델은 assets/, 검사 이력은 data/에 분리한다.



그림 1. 유효 요청의 처리 경로. 기본·통합 결과의 저장 완료를 확인한 뒤 PLC 로 전달한다.

연결 대상	역할과 읽는 기준
웹 /line·/history	저장된 SQLite·원본·결과를 기준으로 이력 조회가 분기된다.
PLC·LED·컨베이어	PLC 가 전달된 Result/Done 을 운전 로직과 연결한다. 통신 관측과 실제 장치 동작 관찰은 별도 근거다.
기본·참고 판정	기본 부품 존재·결품이 최종 판정을 결정한다. 오조립 참고 결과는 별도 기록하며 최종 PASS/FAIL 을 바꾸지 않는다.

근거: [G3, G4, G7, G9], [R05] 3·8 쪽, [R06] 5 쪽: 검사·저장·PLC 게시 순서.

02 Jetson 실행 코드 맵

"Jetson 실행 코드"는 카메라·검출·판정·저장·PLC·웹의 실행 경로 전체다. 아래 경로는 코드 저장소 루트 기준이며, 부록 A 의 고정 커밋 링크로 연결한다.

실행 소스 / 문서	확인할 내용
jetson/standalone_service.py	운영 진입점. config·line profile·PLC 계약을 검증하고 runtime 및 웹 앱을 시작
jetson/src/vision/ detector_factory.py	manifest backend 검증 후 TensorRT 또는 PyTorch adapter 생성. 암묵 fallback 없음
jetson/src/vision/ tensorrt_native.py	TensorRT engine I/O 검증, CUDA buffer·stream, execute_async_v3, H2D/D2H 및 동기화
jetson/src/vision/ tensorrt_preprocess.py	640 정사각 letterbox, padding 114, BGR→RGB, CHW batch1, /255
jetson/src/vision/ tensorrt_postprocess.py	출력 shape/dtype 검증, confidence, class-aware NMS, 원본 좌표 복원
jetson/src/vision/ engine_inspector.py	M2 → pose → rectify → M1 → slot 판정과 stage timing 기록
jetson/src/runtime/ standalone.py	같은 요청의 기본 결과 및 통합 최종판정 저장 완료를 확인한 뒤 PLC 게시 수행
jetson/INSTALL_OPERATING_KO.md	현재 운영본을 다른 Orin Nano 에 설치·검증하는 공식 절차
jetson/products/*/manifest.json	모델 파일, SHA, 입력/출력, 전처리·후처리 계약

명시적인 실행 경로
검출기는 manifest 의 backend 에 따라 선택한다. 운영 TensorRT 경로가 실패했을 때 PyTorch 로 자동 대체하지 않으므로, 실제 실행 모델과 backend 를 메타데이터로 구분할 수 있다.

서비스 시작 전의 검증

- standalone_service.py 는 package·station·line_profile 경로가 장치 로컬 범위인지 확인한다.
- PLC IP 와 local IP, profile mode, 최소 저장공간, bounded deadline 등 현장 계약을 확인한다.
- --check-only 는 실제 서비스 운전 전 파일·package·설정·모델 의존성 확인에 사용한다.

근거: [G2, G4-G9], [R06] 5 쪽: 서비스 구성과 저장 완료 확인 코드.

03 모델 계보: 학습 PT 에서 운영 TensorRT 까지

모델 이름만 맞추는 것이 아니라 export·build·제품 manifest 의 SHA-256 을 따라 실제 운영 파일을 식별한다.

학습	교환 형식	장치 실행	운영 패키지
best.pt	ONNX export	TensorRT plan	manifest + release
원본 가중치	opset 12 640×640, batch 1	TensorRT 10.3 혼합 정밀도	명시적인 backend 모델·설정 연결

그림 2. 학습 PT → ONNX → TensorRT plan → Jetson 운영 패키지의 연결.

단계	M1 부품 검출	M2 엔진 전체 검출
PT SHA-256	42db183006ad2a4f61dc9c22e7a61a5b 506cdacb5099004fc7c3b73b74bf5259	cf117defe782e1ae8b355e0ed13c014f c6f1538aa2dcdbe0fb6cfa54200f7755
ONNX SHA-256	433b16fd1e802bdf2cb0afba6bb6487a 839d94b5adc286185b93fd3953b9c296	5cb024bb911f9357e46c12c3d2a7e4b5 f19af20e5692013037c9235e0deca830
TRT plan SHA-256	743ce54b4d5c7b8a36addf3dbff50fa6 93ec4a509b31b476f68efc69f1bf2d6c	c82f2658ac20d06985a647c15e551c68 1e394451cfa1c53baed64f639bcebf83
운영 package	dynamic_parts_005	envelope_v007_fp16
입력 / 출력	[1,3,640,640] / [1,7,8400]	[1,3,640,640] / [1,5,8400]

변환 계보의 확인 범위
export 기록 → build_manifest → 제품 manifest → 릴리스 및 기존 Jetson 관측의 해시를 대조한 연결이다. 위 SHA-256 은 두 줄을 이어 붙인 64 자리이며 수치나 바이트를 변경하지 않았다.

두 TensorRT plan 은 기존 코드 배포에 실제 바이너리로 포함된 모델이다. 다른 GPU·TensorRT 환경으로 옮길 때는 호환성을 확인하고 필요에 따라 원본 모델로 재빌드·재검증한다.

근거: [G8], [R03] 13 쪽, [R04] 2·15 쪽: 학습·변환·운영 파일의 해시 연결.

04 기록된 ONNX·TensorRT 변환 조건

아래 값은 기존 export 및 build 기록의 설정이다. 재현 시 실제 모델 해시와 실행 환경을 함께 확인한다.

변환 구분	기록된 조건
PT → ONNX	Ultralytics YOLO.export 8.4.146, imgsz=640, batch=1, dynamic=False, half=False, nms=False, simplify=False, opset=12
ONNX → TensorRT	TensorRT 10.3 Builder API, workspace 1 GiB, optimization_level=3, max_aux_streams=0, TF32=False, precision_constraints=OBEY
M1 정밀도	FP16 + head FP32. build 기록에서 FP32 지정 레이어 77 개
M2 정밀도	FP16 + backbone FP32. build 기록에서 FP32 지정 레이어 208 개
I/O	TensorRT engine 입력·출력은 float32 로 검증
운영 파일	새 엔진을 임의로 재빌드하지 않고 현재 제품 package 의 .plan 을 사용

변환 기록을 대조하는 순서

1) 학습 best.pt 의 SHA 를 확인한다.

2) export 기록의 source PT SHA 와 생성 ONNX SHA 를 확인한다.

3) build_manifest 에서 ONNX 입력과 생성 plan SHA, TensorRT 설정을 확인한다.

4) 제품 manifest 의 model artifact SHA 와 plan 을 대조한다.

5) release.json 과 실제 Jetson 관측 해시로 현재 운영 파일까지 연결한다.

혼합 정밀도와 실행 메모리 효율
M1 은 FP16 + head FP32, M2 는 FP16 + backbone FP32 이며 I/O 는 float32 다. 이 구성의 대표 성과는 파일 크기 축소가 아니라, 동일 Jetson 비교에서 관측한 프로세스 RSS 감소다.

FP32 지정 레이어 77 개·208 개는 빌드 설정에 기재된 목록 수다. 실행 중 레이어별 정밀도 프로파일링 결과로 확대해서 해석하지 않는다.

근거: [R04] 2 쪽: provenance/M1·M2 의 export·build_manifest·manifest 기록. [R03] 13 쪽.

05 입력 전처리 계약

TensorRT 경로는 입력의 의미가 달라지지 않도록 정적 640 letterbox 계약을 코드로 고정한다. tensorrt_preprocess.py 의 계약을 어기면 INPUT_CONTRACT_MISMATCH 로 실패한다.

항목	운영 계약
원본	uint8 HWC BGR, 3 채널
resize	gain=min(640/H, 640/W), 선형 보간
letterbox	640×640 중앙 배치, padding value 114
채널/형태	BGR→RGB, HWC→CHW, batch dimension 추가
정규화	float32 또는 선언 dtype 으로 변환 후 /255
메모리	contiguous array
형상 기록	original_shape, resized_shape, gain, left/top/right/bottom padding 을 ImageTransform 에 보존

핵심 처리의 요약

```
# 입력 계약 설명용 요약 (독립 실행 스크립트 아님)
gain = min(640 / height, 640 / width)
padded = cv2.copyMakeBorder(..., value=(114, 114, 114))
rgb_chw = padded[:, :, ::-1].transpose(2, 0, 1)[None]
tensor = np.ascontiguousarray(rgb_chw, dtype=dtype)
tensor /= 255
```

같은 입력이라는 조건
PT 와 TensorRT 에서 imgsz=640 이라는 설정만 같아서는 충분하지 않다. resize·padding·채널 순서·정규화까지 같은 계약을 적용해야 출력 차이를 비교할 수 있다.

근거: [G6] tensorrt_preprocess.py. [R04] 4 쪽: E03 원본 1280×720 의 공통 letterbox 조건.

06 출력 후처리와 좌표 복원

tensorrt_postprocess.py 는 raw head 의 출력 이름·shape·dtype·finite 여부를 검사한 뒤 confidence → class-aware NMS → 원본 좌표 복원 순서로 처리한다.

단계	검증·처리 내용
출력 계약	manifest 의 tensor 이름·shape·dtype 과 실제 반환값이 정확히 일치해야 함
confidence	각 후보에서 argmax class 와 best score 를 선택하고 선언 threshold 보다 큰 후보만 사용
NMS	같은 클래스끼리 IoU 기준으로 중복 제거, max_det 300
좌표 복원	letterbox padding 을 빼고 gain 으로 나눈 뒤 원본 W×H 범위로 clip
비정상 좌표	복원 후 면적이 0 이거나 비정상 값이면 DetectorError
진단 메타데이터	raw candidate, eligible, kept, suppressed, padding/gain, confidence/NMS threshold 저장 가능

기준 운영 NMS IoU 는 0.45 다. 같은 클래스의 중복 예측을 정리하는 NMS IoU 와, 정답 박스에 대한 성능을 계산하는 정답 매칭 IoU 는 목적이 다른 값이다.

모델 검출과 제품 판정의 구분
M1·M2 의 박스 검출은 최종 제품 판정의 입력이다. 제품 판정에는 자세 추정·정렬, 부품 자리 대응과 세 프레임 집계가 추가된다.

Benchmark 후처리의 적용 범위

confidence 0.25 의 시간·검출 건수 비교에는 두 백엔드에 같은 운영 조건을 적용했다. mAP 용 confidence 0.001 에서는 면적 0 후보를 보존하는 평가 전용 연결을 사용했으므로, 이를 운영 후처리와 완전히 같은 경로라고 표시하지 않는다.

근거: [G6] tensorrt_postprocess.py. [R04] 8·12 쪽: confidence/NMS/정답 IoU 와 평가 전용 보완의 구분.

07 Native TensorRT 실행 경로

NativeTensorRTExecutor 는 engine 의 I/O 를 로드할 때 이름·개수·shape·dtype·TensorIOMode·DEVICE 위치·LINEAR format 을 검증한다. 동적/잘못된 차원 또는 예상하지 않은 I/O 는 실행 전에 거부한다.

순서	실행 내용
1. Host 준비	contiguous 입력을 pinned host buffer 에 복사
2. H2D	CUDA non-default stream 에서 device input 으로 비동기 복사
3. enqueue	context.execute_async_v3(stream_handle=...) 실행
4. synchronize	GPU 작업 완료를 동기화하고 완료 시점을 기록
5. D2H	output device → host 비동기 복사 후 다시 동기화
6. 후처리	NumPy output 을 복사해 TensorRTDetector/postprocess 로 전달
7. 메타데이터	engine SHA, TensorRT version, I/O, execute_count, last_timing 을 JSON 가능 구조로 제공

실행과 출력 전송의 완료 확인

```
# NativeTensorRTExecutor 실행 순서 요약
execute_async_v3(stream_handle=self.stream)
_synchronize()
copy_async(output_host, output_device, ..., D2H)
_synchronize()
return {output_name: output_array.copy()}
```

코드의 last_timing 은 host 관측 구간이며 순수 GPU kernel time 이라고 명시하지 않는다. Benchmark 도 별도 GPU kernel 단독시간을 측정하지 않았다.

실행 결과의 식별
engine SHA, TensorRT 버전, I/O 계약, 실행 횟수와 timing 메타데이터를 함께 남길 수 있다. 파일이 존재한다는 확인과 실제 추론이 완료됐다는 확인을 분리한다.

근거: [G6] tensorrt_native.py. [R04] 4 쪽: GPU 실행 및 CPU 출력 완료를 포함한 측정 구간.

08 M2 → Pose → M1 → 3 프레임 판정

engine_inspector.py 는 두 검출기를 독립적으로 호출하는 수준을 넘어서, 엔진 전체 검출을 부품 검출의 좌표계 정렬에 사용한다.

처리 단계	실제 코드의 역할
M2 envelope	현재 frame 에서 product_envelope 검출
pose	유효한 product 가 1 개일 때 EnginePoseEstimator 로 자세 추정
rectify	pose 가 reliable 이면 canonical image 로 변환
M1 parts	canonical frame 에서 필수 부품 검출
restore	부품 검출 좌표를 pose matrix 로 원본 camera 좌표에 복원
slot decision	엔진 전체·부품 detections 를 inspect_frame 에 전달하여 자리 상태 판단
3-frame aggregate	세 프레임의 자리별 상태가 일치하는지 확인해 PASS/FAIL/REVIEW/ERROR 확정

engine_inspector 는 product_detection_ms, pose_ms, rectification_ms, part_detection_ms, slot_decision_ms 와 engine_processing_ms 를 단계별 진단값으로 남길 수 있다.

최종판정 저장과 PLC 게시의 연결

구간	확인·처리
기본 결과	_commit_and_publish()에서 같은 요청의 완료 기록과 storage_success 를 확인한다.
통합 최종 저장	OPENER_CYCLIC 경로의 submit_final()로 저장을 요청한다.
저장 완료 대기	_poll_final_commit()에서 완료 응답과 요청 문맥을 검사한다.
PLC 게시	확인된 결과만 _publish_committed()로 게시한다. Result 이후 Done 순서의 전달과 연결된다.

검사 파이프라인의 의미
최종 PASS/FAIL 은 YOLO 단독 호출의 결과가 아니다. 자세 정렬·부품 자리 대응·3 프레임 확정 및 결과 저장을 거쳐 장치 전달로 연결한다.

근거: [G7, G9], [R05] 3·7 쪽, [R06] 5 쪽: 기본 판정과 통합 결과 저장·게시 경로.

09 현재 운영본 설치·기동 절차

다른 Jetson 에는 개발 소스를 재조합하지 않고 portable/operating_runtime.tar.gz 와 고정 release 를 기준으로 설치하도록 문서화돼 있다. 기존 setup.sh/install.py 의 CORE MOCK 경로와 구분한다.

아래는 신규 장치 설치의 순서와 대표 명령이다. 전체 옵션·경로·환경 준비는 [G2] 설치 안내를 따르며, 기존 운영 장치에 그대로 덮어쓰는 명령 목록이 아니다.

단계	대표 명령 / 확인할 내용
1. 설치 입력 검증	python3.10 -B -X utf8 jetson/install_operating.py --verify
2. 새 장치 폴더 준비	install_operating.py --prepare --camera <by-id> --station-id <id>
3. Python 환경	python3.10 -m venv --system-site-packages ~/oned_device_bench/envs/app_v1
4. 운영 점검	check_operating.py standalone_service.py --check-only check_operating.py --gpu-smoke
5. 기준 등록	launch_live.py 로 STOP 상태에서 기준 엔진과 검사 영역 등록
6. PLC 활성화 전 확인	--enable-plc --confirm-stopped --confirm-camera-setup --confirm-plc-contract
7. systemd	engine-inspector.service 등록 후 enable --now, status/journal 확인
8. 현장 인수 확인	정상·알려진 결품으로 원본 3 장, 판정·저장, Result/Done, LED·컨베이어 재이송을 확인

장치 영향과 인수시험을 분리
서비스 등록·기동·PLC 활성화는 실제 장치에 영향을 주는 단계다. 설비 정지 상태와 현장 설정을 확인한 뒤 수행한다. 파일 검증이나 GPU smoke 성공만으로 실물 인수시험까지 완료됐다고 보지는 않는다.

현장 구성은 푸셔 없는 라인이다. 정상·NG 에 따른 결과 표시와 운전 조건에서의 재이송을 확인하며, 푸셔를 이용한 자동 분류·배출 기능으로 설명하지 않는다.

근거: [G2, G3], [R05] 8 쪽: 설치 절차, 저장 PLC 로직 및 운영자 관찰.

10 최적화 결과와 운영 적용 확인

같은 Jetson 에서 E03 72 장으로 원본 PT FP32 와 운영 TensorRT 를 비교했다. 아래 원본 수치의 화살표는 PT → TensorRT 다.

항목	M1 부품 검출	M2 엔진 전체 검출
평균 지연시간	29.313 → 29.022 ms	29.480 → 30.266 ms
처리량	34.06 → 34.41 image/s	33.87 → 33.00 image/s
프로세스 RSS 관측 최대	1,250.5 → 636.9 MiB (약 49.1% 감소)	1,263.8 → 649.7 MiB (약 48.6% 감소)
고정 조건 TP/FP/FN	PT·TRT 모두 192/0/0	PT·TRT 모두 72/0/0
mAP50-95	0.639911 → 0.640873	0.836717 → 0.836629

대표 성과: 실행 프로세스 메모리 절감
프로세스 RSS 관측 최대는 M1 약 49.1%, M2 약 48.6% 감소했다. 이는 라이브러리·입력까지 포함한 프로세스 RSS 이며 GPU 전용 메모리나 모델 파일 크기의 감소율은 아니다.

평균 지연시간은 M1 약 1.0% 감소, M2 약 2.7% 증가했다. 두 모델 모두 약 30ms 의 개별 처리시간을 기록했으며, 이번 비교에서는 큰 가속보다 메모리 절감 효과가 뚜렷했다.

반복 조건과 결과 해석

구분	조건
본 측정 횟수	4 개 조건 × 3 회 독립 실행 × 72 장 × 5 회 순환 = 4,320 개 지연시간 기록
시간 포함 범위	메모리의 원본 → 전처리 → GPU 실행·CPU 출력 완료 → 후처리. 디스크 읽기·디코딩·예열 등 제외
고정 검출 건수	confidence 0.25, NMS IoU 0.45, 정답 매칭 IoU 0.50
mAP 비교	confidence 0.001 의 별도 평가 연결. E03 개발 검증이며 독립 최종시험·제품 판정 성능과 구분

근거: [R04] 4-8·12 쪽: 반복 측정 조건, 원본 비교 수치 및 평가 전용 연결.

11 구현 강점과 적용 범위

구현 요소	평가자가 확인할 수 있는 강점
모델 계보	학습 checkpoint 부터 ONNX, TensorRT, 제품 package, 장치 release 까지 SHA-256 연결
명시적 backend	manifest 계약에 따라 adapter 를 선택하고 silent fallback 금지
고정 입출력 계약	전처리·출력 shape/dtype·NMS·좌표 복원 조건을 코드로 검사
단일 운영 supervisor	systemd 서비스 하나가 카메라·추론·PLC child·웹을 감독
복합 검사 파이프라인	M2 엔진 검출과 자세 정렬 후 M1 부품 검출, 3 프레임 일관성으로 기본 판정
저장·게시 연결	동일 요청의 기본·통합 결과 저장 완료를 확인한 뒤 PLC 로 전달
재현·운영 분리	portable 설치, check-only, GPU smoke, 현장 기준 등록, PLC 활성화를 단계적으로 분리
정량 검증	동일 Jetson Benchmark 와 실물 7 건 통합시험·DB 조회 증빙을 별도 보존

기존 시험본과 후속 운영본

실물 7 회 시험은 4c4b935f8ab0404a 실행본에서 수행했다. 이후 32c484ac6aeb07f1 은 두 UI 파일의 변경 범위를 별도로 확인한 운영본이다. 같은 모델 계보를 사용한다는 설명과, 후속 버전에서 실물시험을 다시 했다는 설명은 구분한다.

적용 범위와 후속 검증

- E03 72 장은 개발 검증 자료다. 독립 최종시험과 다양한 현장 조건 검증은 별도 범위다.
- 모델별 단독 처리시간·RSS 와 전체 검사·PLC 응답시간은 다른 지표다. TensorRT 가 모든 조건에서 더 빠르다는 결과는 아니다.
- 신규 장치의 카메라·PLC·기준 등록과 실물 인수시험이 필요하며, 다른 GPU·TensorRT 환경은 호환성 및 재빌드 필요성을 확인한다.

근거: [G2-G9], [R04] 13-14 쪽, [R05] 12-13 쪽: 구현 범위와 운영·시험 버전 연결.

부록 A 실행·변환 소스 찾아보기

코드·학습 소스 저장소와 문서·증빙 저장소를 분리한다. 아래 소스 링크는 평가에 사용한 기존 코드 커밋으로 고정되어 있으며, 현재 브랜치 HEAD 나 장치 배포 ID 를 뜻하지 않는다.

Engine_Assembly_Defect_Inspector · 평가 근거 코드

cfaa372eece0cd6a7dc659cc316c256fb4b6af76

ID	소스 링크	확인할 내용
G1	기준 소스 스냅샷	평가 코드의 기준 커밋
G2	jetson/INSTALL_OPERATING_KO.md	portable 설치·환경·GPU smoke·현장 설정
G3	jetson/INTEGRATED_RUNTIME.md	systemd·OpENer·런타임·Journal 구조
G4	jetson/standalone_service.py	설정·장치 계약 검증 및 런타임 구성
G5	jetson/src/vision/detector_factory.py	명시적인 backend 선택과 자동 대체 금지
G6-a	jetson/src/vision/tensorrt_native.py	native 실행·CUDA 버퍼·동기화
G6-b	jetson/src/vision/tensorrt_preprocess.py	입력·letterbox·채널·정규화 계약
G6-c	jetson/src/vision/tensorrt_postprocess.py	출력·NMS·좌표 복원 계약
G7	jetson/src/vision/engine_inspector.py	M2·Pose·Rectify·M1·자리 대응
G8	jetson/products/README.md	운영 모델·manifest·환경 의존성
G9	jetson/src/runtime/standalone.py	기본·통합 저장 완료 확인 및 PLC 게시

각 모델의 제품 manifest

M1 · dynamic_parts_005/manifest.json

M2 · envelope_v007_fp16/manifest.json

G6 은 G6-a·b·c 를 함께 가리킨다. 화면에 나뉘어 보이는 소스 경로는 링크를 통해 전체 경로로 확인할 수 있다.

부록 B 세부 보고서와 근거 연결

문서·증빙 저장소: Engine_Assembly_Runtime_Review. 아래 링크는 기존에 확보한 보고서 버전의 보존 스냅샷을 가리킨다. 보고서별 세부 근거 위치를 함께 제시한다.

Engine_Assembly_Runtime_Review · 산출물 안내

ID	보고서 링크	연결할 근거
R03	모델 평가서 v1.1	학습·개발 검증, PT→ONNX→TRT 연결, 혼합 정밀도 13-14 쪽
R04	Benchmark 보고서 v1.1	동일 Jetson 의 속도·메모리·검출 성능, 평가 연결 조건 2-8·12-15 쪽
R05	통합 테스트 결과서 v1.0	기본·참고·통합 저장, PLC 응답, 7 회 시험과 후속 UI 운영본 3·7-8·11-13 쪽
R06	DB 구조·조회 방식 v1.0	같은 요청의 저장 완료 확인 후 게시하는 코드와 이벤트 순서 5·7·12-14 쪽

원문 자료를 찾는 기준

자료 종류	기존 자료의 연결 위치
변환·모델 계보	모델 평가서 부록 A·D 및 Benchmark 보고서 부록 A·B. export·build_manifest·제품 manifest 와 모델 해시를 대조한다.
실물·저장·PLC 기록	통합 테스트 결과서 부록 C 및 DB 설명서 부록. inspection_id 로 원본·기본 결과·통합 결과·완료 이벤트를 연결한다.
설치 전체 절차	부록 A 의 G2 설치 안내를 우선 사용한다. 본문의 대표 명령만으로 신규 장치 설정을 생략하지 않는다.

문서 개정 범위

v1.1 은 기존 v1.0 의 처리 흐름 그림, 저장·PLC 연결 설명, 평균 지연시간 표현, 표·한글 가독성과 근거 링크를 정리한 문서 개정이다. 학습·측정 수치와 모델 해시는 보존했다.

문서 작성 과정에서 운영 장치 접속, 서비스 재시작, PLC 운전, 새 추론·모델 변환·현장시험은 수행하지 않았다. 기존 evidence 파일도 재생성하거나 재압축하지 않았다.

코드 기준: cfaa372eece0cd6a7dc659cc316c256fb4b6af76
보고서 링크의 보존 기준: 3c8273052e629a356e322419d692db0cc00ff156